

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Cloud Image Registry & Distribution Data
Class Applied Computing Name Lu Moyou ID 202483910013

一、Experimental purpose

Practice Version Control: Learn to use Docker Tags to manage different iterations of a cloud application.

Validate Decoupled Deployment: Simulate a disaster recovery scenario to verify the ability to deploy services in a remote environment without source code.

Security Best Practices: Implement Secret Management using Personal Access Tokens (PAT) instead of plaintext passwords.

二、Experiment Contents

In this lab, you will transition from a "Standalone Developer" to a "Cloud Engineer." You will:

- Configure secure authentication with GitHub Container Registry (GHCR).
- Tag and push your web application image to the cloud.
- Perform a version upgrade (v1.0 to v2.0) and observe the

Layered Storage mechanism.

- Perform a "Clean - room Deployment" by pulling your image onto a fresh environment after wiping local data.

三、 Detailed steps of the experiment

Lab Report Requirements: Clearly present the procedure step by step, with corresponding screenshots provided for each stage. **Screenshots must demonstrate that they were taken on your own computer**, for example by including visible system time, desktop environment, or other identifiable personal elements.

Step 1: Secure Authentication (PAT Generation)

Cloud platforms use Tokens for automation and security.

1.1 Generate Token

Navigate to GitHub Settings -> Developer settings -> Personal access tokens -> Tokens (classic).

1.2 Permissions

Click Generate new token (classic). Select the scope write:packages (this automatically includes repo and read:packages).

1.3 Save Token

Copy the generated token immediately. You will not be able to see it again.

1.4 Login via Terminal

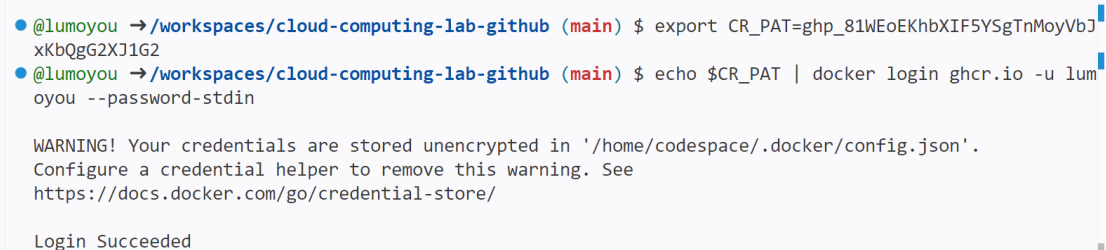
In your Codespaces terminal, run and **explain** the following code:

```
export CR_PAT=YOUR_COPIED_TOKEN  
  
echo $CR_PAT | docker login ghcr.io -u YOUR_GITHUB_USERNAME  
  
--password-stdin
```

Observation: You should see Login Succeeded.

Note: You may see a warning about "credentials stored unencrypted."

This is normal in a Cloud IDE environment like Codespaces. As long as you see "Login Succeeded," you are ready for Step 2.



```
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ export CR_PAT=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  
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ echo $CR_PAT | docker login ghcr.io -u lumoyou --password-stdin  
  
WARNING! Your credentials are stored unencrypted in '/home/codespace/.docker/config.json'.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/go/credential-store/  
  
Login Succeeded
```

Step 2: Tagging for the Cloud

Docker images require a specific naming convention to be recognized by cloud registries.

2.1 Build the Docker Image cloud-lab-image:v1 based on lab 2.

2.2 Apply Tag: Convert your local image into a cloud-ready format.

Note: Your GitHub username must be in LOWERCASE

docker tag cloud-lab-image:v1

ghcr.io/YOUR_USERNAME/cloud-app:v1.0

```
@lumoyou →/workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-lab-image:v1 .
=> [internal] load build definition from Dockerfile                                0.0s
=> => transferring dockerfile: 311B                                              0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine                  2.4s
=> [auth] library/nginx:pull token for registry-1.docker.io                     0.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                    0.0s
=> [internal] load build context                                                 0.0s
=> => transferring context: 904B                                                  0.0s
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816 2.8s
=> => resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816 0.0s
=> => sha256:fbaed3f7fcbe6a0d591ac8601934f1f05252cee42741fde9e950dce55580af 20.28MB / 20.28MB 0.7s
=> => sha256:a5008f4a4b257356728e2feaf2b5b9e2054c0c577d324e76abbf14470b5f1cfd 1.40kB / 1.40kB 0.7s
=> => sha256:e654dbbbb9e15a1fa035d24728aeeb60bc655f4ebd4d6215a496b77a7d104697 1.21kB / 1.21kB 0.7s
=> => sha256:94d083cf706ab544ec7e9bcaba5c164db87b3d3a56176bf2fca440d535c16b0d 405B / 405B 0.7s
=> => sha256:43f834d60d8af3f133c7e76a202d28cd62cc026a561edca72ee752ef01bfaacd 628B / 628B 0.2s
=> => sha256:de1b677d8c003ce9e558f3a0cd4dec3c035f424ecddd68063043a593ad572257 957B / 957B 0.3s
=> => sha256:abaae85d1626e429b3f1209aea369c0af9562cc06b5e075c006e0f699bba35f2 1.88MB / 1.88MB 0.6s
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB 0.6s
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 0.2s
=> => extracting sha256:abaae85d1626e429b3f1209aea369c0af9562cc06b5e075c006e0f699bba35f2 0.2s
=> => extracting sha256:43f834d60d8af3f133c7e76a202d28cd62cc026a561edca72ee752ef01bfaacd 0.1s
=> => extracting sha256:de1b677d8c003ce9e558f3a0cd4dec3c035f424ecddd68063043a593ad572257 0.0s
=> => extracting sha256:94d083cf706ab544ec7e9bcaba5c164db87b3d3a56176bf2fca440d535c16b0d 0.0s
=> => extracting sha256:e654dbbbb9e15a1fa035d24728aeeb60bc655f4ebd4d6215a496b77a7d104697 0.0s
=> => extracting sha256:a5008f4a4b257356728e2feaf2b5b9e2054c0c577d324e76abbf14470b5f1cfd 0.0s
=> => extracting sha256:fbaed3f7fcbe6a0d591ac8601934f1f05252cee42741fde9e950dce55580af18 0.6s
=> [2/2] COPY index.html /usr/share/nginx/html/index.html                     1.2s
=> exporting to image                                                            0.2s
=> => exporting layers                                                            0.1s
=> => exporting manifest sha256:6810914ef5eee44b07a3fb4542a0808e59aab5588de3dc0526744e47466ee 0.0s
=> => exporting config sha256:83df2dd294b29c0a177e7e9d0bec6f44dc3f44a00968a871bd87b9fc986b949 0.0s
=> => exporting attestation manifest sha256:60f011b88f5661a6d74a384075e04e67690f9304e479ea475 0.0s
=> => exporting manifest list sha256:1022b1b2ef02e9e902654bc537de3011755dfc5ed6ab7527e73c2cbf 0.0s
=> => naming to docker.io/library/cloud-lab-image:v1                          0.0s
=> => unpacking to docker.io/library/cloud-lab-image:v1                        0.0s

@lumoyou →/workspaces/cloud-computing-lab-github (main) $ docker images | grep cloud-lab-image
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
cloud-lab-image:v1 1022b1b2ef02 92.7MB 26MB
@lumoyou →/workspaces/cloud-computing-lab-github (main) $ docker tag cloud-lab-image:v1 ghcr.io/lumoyou/cloud-app:v1.0
```

Step 3: Pushing the Image (The "Ship" Phase)

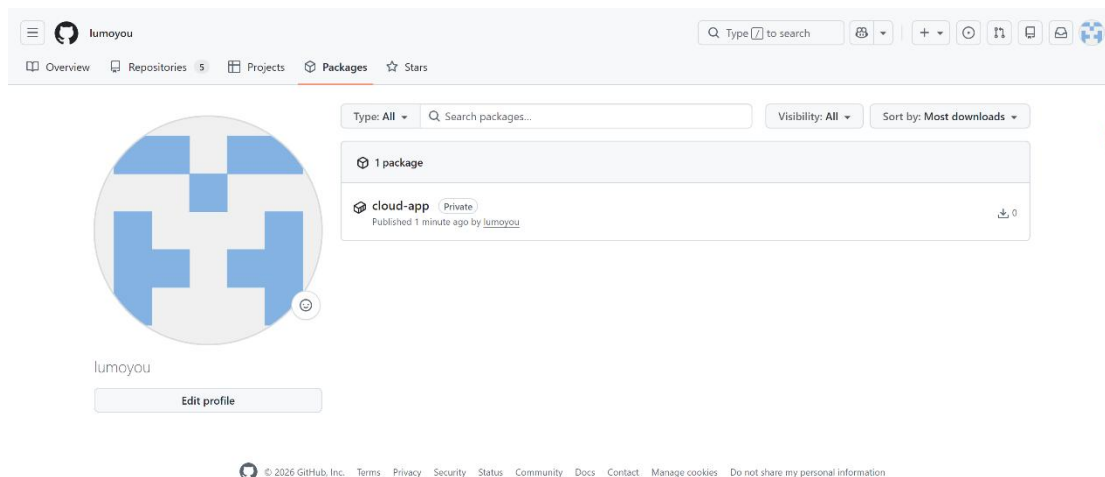
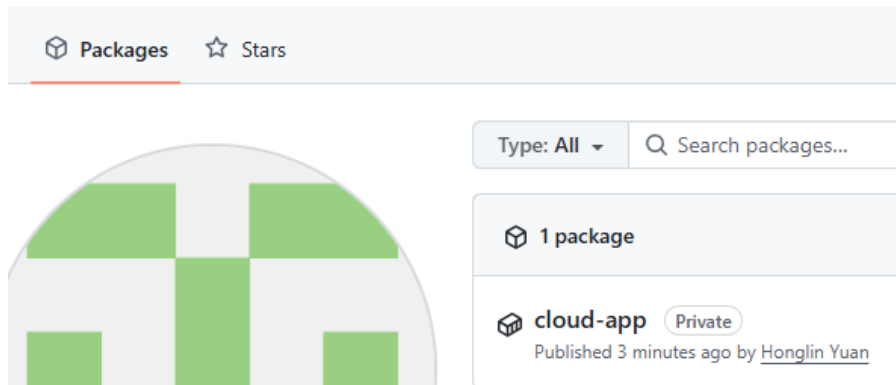
Upload your packaged application to GitHub's global infrastructure.

3.1 Push:

docker push ghcr.io/YOUR_USERNAME/cloud-app:v1.0

3.2 Verify:

Go to your GitHub Profile -> Packages. You should see cloud - app listed.



Step 4: Iteration & Layer Analysis (v2.0)

Cloud computing relies on incremental updates.

4.1 Modify Code

Open index.html and change a visible element (e.g., change the background color or update text to "Welcome to Version 2.0").

4.2 Rebuild & Retag

`docker build -t cloud-lab-image:v2 .`

`docker tag cloud-lab-image:v2`

`ghcr.io/YOUR_USERNAME/cloud-app:v2.0`

4.3 Push v2.0: Run docker push

ghcr.io/YOUR_USERNAME/cloud-app:v2.0

Observation: Notice that some lines say Layer already exists. This is Docker's efficiency—only the changed "layer" (the HTML) is uploaded, saving time and bandwidth.

```

@lumoyou → /workspaces/cloud-computing-lab-github (main) $ docker images | grep cloud-lab-image
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
cloud-lab-image:v1          1022b1b2ef02      92.7MB      26MB
cloud-lab-image:v2          4be94c03ee16      92.7MB      26MB
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ docker tag cloud-lab-image:v2 ghcr.io/lumoyou/cloud-app:v2.0
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ docker images | grep ghcr.io/lumoyou/cloud-app
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/lumoyou/cloud-app:v1.0  1022b1b2ef02      92.7MB      26MB
ghcr.io/lumoyou/cloud-app:v2.0  4be94c03ee16      92.7MB      26MB
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ docker push ghcr.io/lumoyou/cloud-app:v2.0
The push refers to repository [ghcr.io/lumoyou/cloud-app]
fbaed3f7fcbe: Layer already exists
accde706a59a: Pushed
e654dbbbb9e1: Layer already exists
94d083cf706a: Layer already exists
a5008f4a4b25: Layer already exists
d05080286492: Layer already exists
de1b677d8c00: Layer already exists
6a0ac1617861: Layer already exists
abaae85d1626: Layer already exists
43f834d60d8a: Layer already exists
v2.0: digest: sha256:4be94c03ee166eb8a7ca983629c20865a934bee3960c2b67ea06932c158f406c size: 856

```

Step 5: Disaster Recovery (The "Pull" Test)

Prove that your application is now independent of your local machine.

5.1 Wipe Local Data: Delete all local images and containers to simulate a new server.

Warning: This removes all local images

docker system prune -a

5.2 Remote Deployment: Deploy directly from the cloud without any source code files.

docker run -d -p 8888:80

ghcr.io/YOUR_USERNAME/cloud-app:v1.0

5.3 Verify: Access the service via the "Ports" tab on port 8888.

5.4 Run: `docker run -d -p 8888:80 ghcr.io/honglinyuan/cloud-app:v2.0`

You will see Bind for 0.0.0.0:8888 failed: port is already allocated. Find methods to solve this problem.

```
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ docker system prune -a
ibxbi5m39hsk0ezwhlggpx8pe
gmyeovzhdjmtc72br105aje4s
ntgiox8jn4a3isa4gsvkpubqn
fkwbxd5kjubnl8gk18vf5wzss
r4tf4pkssivkjbxda1cmr1msc
thmau7ch9eh28vga7k3tgux2e
i1dcjknolsscmeyau19392tu
6u5rqin0xez8dgyaqs4l703s
l12r1fi5k9l32t7rr6mhjp7eg

Total reclaimed space: 92.78MB
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8888:80 ghcr.io/lumoyou/c
loud-app:v1.0
Unable to find image 'ghcr.io/lumoyou/cloud-app:v1.0' locally
v1.0: Pulling from lumoyou/cloud-app
d05080286492: Pull complete
6a0ac1617861: Pull complete
abaae85d1626: Pull complete
43f834d60d8a: Pull complete
de1b677d8c00: Pull complete
94d083cf706a: Pull complete
e654dbbbb9e1: Pull complete
a5008f4a4b25: Pull complete
fbaed3f7fcbe: Pull complete
1dcf6225bfa6: Download complete
Digest: sha256:1022b1b2ef02e9e902654bc537de3011755dfc5ed6ab7527e73c2cbf76f343a9
Status: Downloaded newer image for ghcr.io/lumoyou/cloud-app:v1.0
f584729b15769c789b0a6f1bea37b372675f71a7af337363d5e1332b5f9b04df
@lumoyou → /workspaces/cloud-computing-lab-github (main) $
```



端口	转发地址	正在运行的进程	可见性	源
8888	https://...	/usr/libexec/docker/docker-proxy -...	Private	自动转发

添加端口

Step 6: Image Optimization - "Standard vs. Alpine" (15 mins)

In cloud computing, image size directly impacts storage costs and deployment speed (Cold Start time).

6.1 Compare Base Images

Create two different Dockerfiles to see the impact of OS selection:

6.1 Create a file named Dockerfile.alpine with the following content:

FROM nginx:alpine

COPY . /usr/share/nginx/html

6.2 Build both versions:

docker build -t cloud-app:standard -f Dockerfile .

docker build -t cloud-app:alpine -f Dockerfile.alpine .

6.3 Size Comparison

Run : docker pull nginx:latest

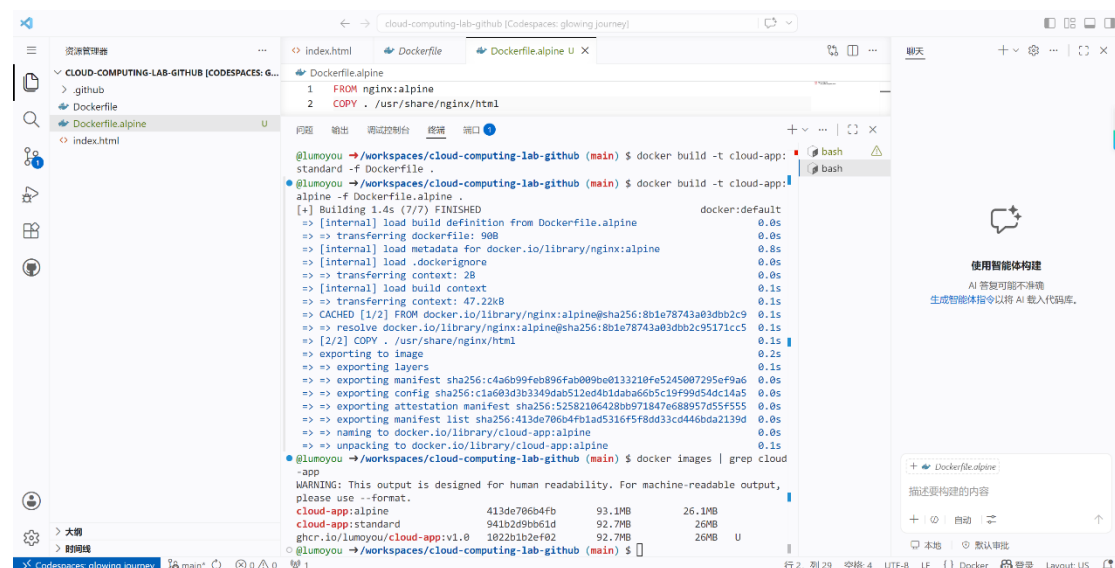
docker pull nginx:alpine

docker images | grep nginx

compare the size of cloud - app:standard vs. cloud - app:alpine.

Report Task: Record the exact sizes in your lab report. Explain why

"Alpine" is the industry standard for production microservices.



The screenshot shows a VS Code editor with a workspace named 'cloud-computing-lab-github'. It contains two Dockerfiles: 'Dockerfile' and 'Dockerfile.alpine'. The terminal shows the following commands and output:

```
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-app:standard -f Dockerfile .
@lumoyou → /workspaces/cloud-computing-lab-github (main) $ docker build -t cloud-app:alpine -f Dockerfile.alpine .
```

The output for the alpine build shows the following steps and sizes:

Step	Size
[+] Building 1.4s (7/7) FINISHED	
⇒ [internal] load build definition from Dockerfile.alpine	0.0s
⇒ ⇒ transferring dockerfile: 98B	0.0s
⇒ [internal] load metadata for docker.io/library/nginx:alpine	0.8s
⇒ [internal] load .dockerignore	0.0s
⇒ ⇒ transferring context: 28	0.0s
⇒ [internal] load build context	0.1s
⇒ ⇒ transferring context: 47.22kB	0.1s
⇒ CACHED [1/2] FROM docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c9	0.1s
⇒ resolve docker.io/library/nginx:alpine@sha256:8b1e78743a03dbb2c95171cc5	0.1s
⇒ [2/2] COPY . /usr/share/nginx/html	0.1s
⇒ exporting to image	0.2s
⇒ ⇒ exporting layers	0.1s
⇒ ⇒ exporting manifest sha256:c4a6b99feb896fab009be0133210fe5245087295eF9a6	0.0s
⇒ ⇒ exporting config sha256:cia683d3b3349dab512ed4b1daba66b5c19f99e54dc14a5	0.0s
⇒ ⇒ exporting attestation manifest sha256:52582106428bb971847e688957d55f555	0.0s
⇒ ⇒ exporting manifest list sha256:413de706b4fb1ad5316f5f8dd33cd446bda2139d	0.0s
⇒ ⇒ naming to docker.io/library/cloud-app:alpine	0.0s
⇒ ⇒ unpacking to docker.io/library/cloud-app:alpine	0.1s

The final output shows the sizes of the images:

Image	Size
cloud-app:alpine	93.1MB
cloud-app:standard	92.7MB
ghcr.io/lumoyou/cloud-app:v1.0	92.7MB

Step 7: Use git to save your changes.

- git add . (Gather all your lab changes)

- `git commit -m "Finished Lab 03 " (Seal the package)`
- `git push` (Ship it to GitHub for permanent storage)

```
git .10/ lumoyou/ cloud-app.v1.0 102201020102 0247110 2010 0
• @lumoyou →/workspaces/cloud-computing-lab-github (main) $ git add .
• @lumoyou →/workspaces/cloud-computing-lab-github (main) $ git commit -m "Finished Lab 03"
[main 9a7b957] Finished Lab 03
1 file changed, 2 insertions(+)
create mode 100644 Dockerfile.alpine

main: see the note about fast forwards in git push help for details.
• @lumoyou →/workspaces/cloud-computing-lab-github (main) $ git push origin main --force
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 2 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (15/15), 4.02 KiB | 4.02 MiB/s, done.
Total 15 (delta 3), reused 3 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To https://github.com/lumoyou/cloud-computing-lab-github
+ 1a71b79...9a7b957 main -> main (forced update)
```

四、Reflection

1. Problems Encountered and Proposed Solutions

Problem 1: Port 8888 already in use when running v2.0 after v1.0.

Solution: Stop the existing container (`docker stop <id>`) or map to a different port (`-p 8889:80`).

Problem 2: `Login Succeeded` warning about unencrypted credentials in Codespaces.

Solution: This is expected in cloud IDEs; no action needed as the environment is ephemeral.

Problem 3: Token not working – forgot to enable `write:packages` scope.

Solution: Regenerate token with correct permissions.

2. Efficiency: Why did Step 4.3 (Pushing v2.0) upload much faster than

Step 3.1? Relate this to the concept of Layer Caching.

Step 4.3 was faster because Docker uses **layer caching**.

When you push v2.0, only the **changed layers** (e.g., the new `index.html`) are uploaded.

The base image layers (e.g., `nginx`, system dependencies) already exist in the registry from the v1.0 push.

This reduces upload time, bandwidth, and storage — a key cloud efficiency.

3. Security: Why is using a Personal Access Token (PAT) more secure

than using your actual GitHub password in a terminal?

Limited scope: A PAT can be restricted to specific permissions (e.g., write:packages), whereas a password grants full account access.

Revocable: PATs can be revoked individually without changing your password.

No credential exposure: The password is never stored or typed into the terminal.

Expiration: PATs can have expiration dates, reducing long-term risk.

Auditability: PATs can be named and tracked per task or script.

Using a PAT follows the **principle of least privilege** — a core security best practice in DevOps.